University of Westminster

School of Computer Science & Engineering

3D Interactive Media Development (5MMCS007C)

Individual Coursework 2026/27

Student name	P.K.D.Himaya Weenadhari
University of Westminster ID	w2182084
IIT student ID	20250288
Programme	BSc(Hons) Computer Science
Tutorial group	G8
Deadline	12 th November 2026

01. Individual contribution

Rabbit Survival is a Unity prototype in which a white rabbit follows a forest trail made of LEGO bricks, avoids patrolling hunters and crosses a bridge to reach a second rabbit inside a cave. The Student 2 allocation was the Tutorial 02/03 mouse input brick controller, in which the player clicks a brick to move it and open a path. My implementation also includes a LegoWalk minifigure with a UI walk button, the carrot collectibles with a counter and heads-up display, and the group scene built in which every member's Unity package was imported. Together, these features give the player a way to interact directly with the world using the mouse, a reason to explore the trail and a single scene that holds the whole group's work.

Mouse input brick controller

The mouse-controlled bricks form a wall that blocks the trail, and the rabbit cannot walk through it or go around it. The player must move the bricks aside to clear a path. Each brick is the object [Student2BrickMaterial] and has a Collider and a Rigidbody component, so it takes part in the physics of the scene and blocks the rabbit until it is moved.

The behavior is in MouseInput.cs, which is attached to each brick. It uses Unity's OnMouseDrag event, which is called while the player holds the left mouse button over an object's collider. This means the collider handles selection. The player clicks the brick they can see, and only that brick responds. Selection and manipulation happen in one action, as in the direct manipulation techniques covered in the 3D-UI lectures.

While the brick is being dragged, the script reads the mouse movement from Input.GetAxis("Mouse X") and Input.GetAxis("Mouse Y"). It multiplies each value by a public speed variable (default 10) and by Time.deltaTime. The result is applied with transform.Translate in Space.World. Horizontal mouse movement moves the brick along the world X axis. Vertical mouse movement moves it along the world Z axis, which pushes the brick across the ground in the same direction as the mouse. The brick is never moved on the Y axis, so it stays on the ground, and the player cannot lift it. Using world space means the movement stays the same even if the brick has been rotated. The speed value is public, so it can be tuned in the Inspector without changing the code.

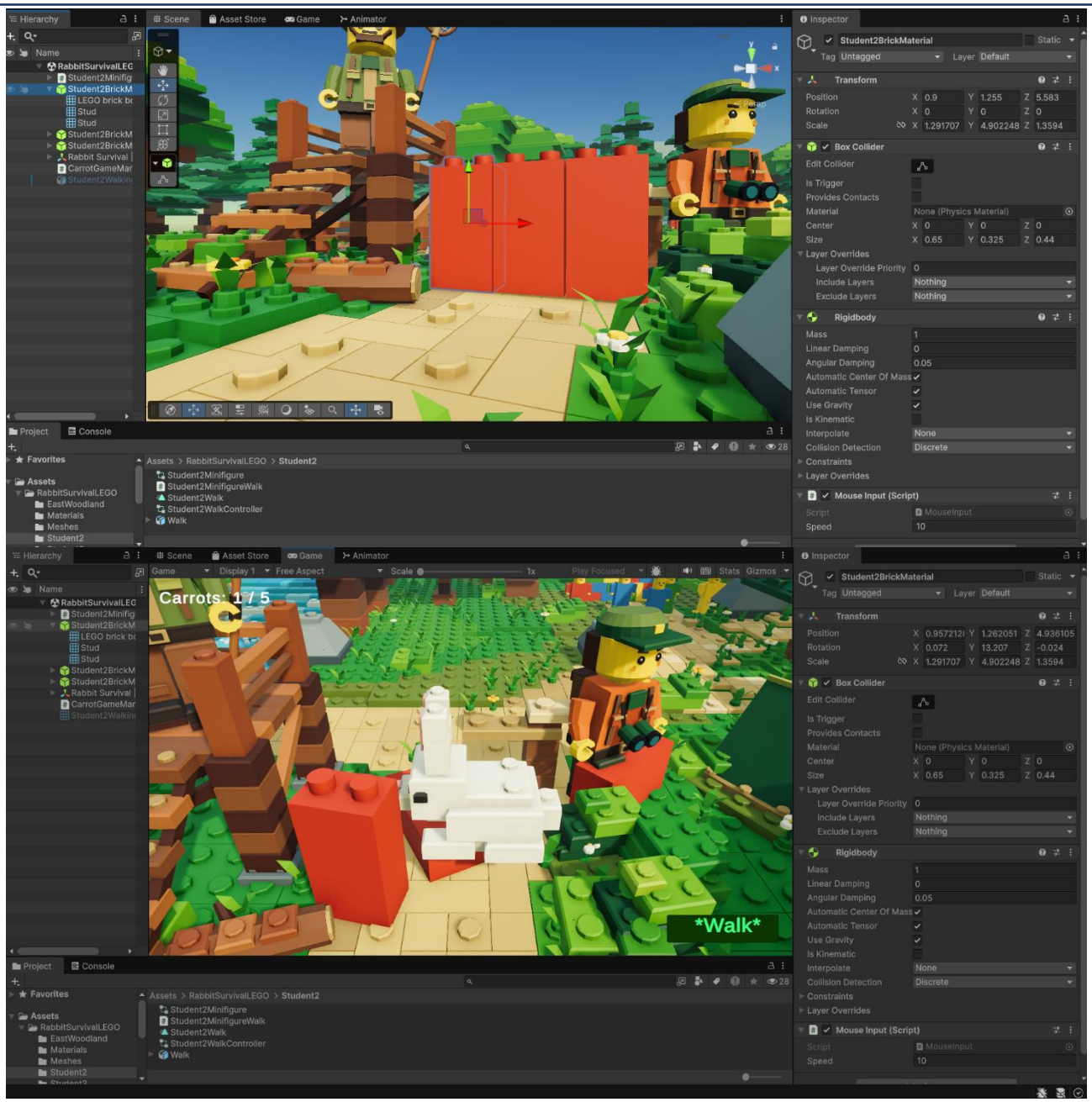
[[MouseInput.cs]] •

Assembly-CSharp | MouseInput | speed

```
1   using UnityEngine;
2
    Unity Script (5 asset references) | 0 references
3   public class MouseInput : MonoBehaviour
4   {
        Unity Serialized Field = 10 | 2 references
5       public float speed = 10f;
6
        Unity Message | 0 references
7       void OnMouseDrag()
8       {
9           float moveX = Input.GetAxis("Mouse X") * speed * Time.deltaTime;
10          float moveZ = Input.GetAxis("Mouse Y") * speed * Time.deltaTime;
11
12          transform.Translate(moveX, 0f, moveZ, Space.World);
13      }
14  }
```

The script uses Unity's legacy Input class. Mouse movement is read with Input.GetAxis("Mouse X") and Input.GetAxis("Mouse Y"), and selection is handled by the built-in OnMouseDrag event. It does not use the new Input System package, so there are no InputAction objects to create, enable or dispose of. The script has no enable or disable code of its own. OnMouseDrag is only called while the MouseInput component is enabled and the player is holding the mouse button over the brick's collider. Disabling the component, or removing the collider, therefore stops the brick from being dragged. Because the legacy axes are polled by Unity every frame, nothing needs to be cleaned up when the brick is destroyed. The project's Active Input Handling setting is set to "Both", so this script can run alongside the new Input System code that other group members used, such as Student 4's key trigger.

The only field exposed in the Inspector is the public speed variable, which defaults to 10. It scales how far the brick moves for each unit of mouse movement, so the group can adjust how heavy or responsive the bricks feel without editing the code. The script has no camera reference or target position. Selection relies on the brick's collider, and the player decides where the brick ends up by dragging it. The brick Collider and Rigidbody components are set up on the object in the Inspector, not through the script.


Hierarchy
Scene
Asset Store
Game
Animator
Inspector
Student2BrickMaterial
Transform
Box Collider
Rigidbody
Mouse Input (Script)
Project
Console
Favorites
Assets
Carrots: 1 / 5
Walk

LegoWalk minifigure and UI walk button

The prototype minifigure uses the lecturer's LegoWalk.fbx model. I animated it with a script, Student2MinifigureWalk.cs, which rotates the limbs directly instead of playing an Animator clip. The script is attached to the minifigure [[Student2Minifigure]] and has four public Transform fields, leftHip, rightHip, leftShoulder and rightShoulder. I assigned these in the Inspector to the matching joints in the model's hierarchy.

When the component loads, Awake calls CacheRestPose, which stores each joint's starting localRotation. Every swing is applied relative to that stored pose, so the figure always returns to its original stance. A Unity UI Button calls the public method PlayWalk through its On Click event, as in Tutorial 05.2. PlayWalk sets a countdown timer, timeLeft, to the duration value of 2.4 seconds. It does not move the limbs itself.

The movement is done in Update. While the timer is above zero, the script reduces it by Time.deltaTime and works out the elapsed time. It then calculates a swing angle with a sine wave: Mathf.Sin(elapsed * cyclesPerSecond * Mathf.PI * 2f) * swingAngle. This gives a smooth back-and-forth motion that peaks at 30 degrees and repeats twice per second, so the 2.4-second walk contains almost five steps. The angle is applied as a rotation around each joint's local X axis. The left hip and right shoulder receive the positive angle, and the right hip and left shoulder receive the negative angle. This makes opposite limbs swing together, as in a natural walking gait. Each assignment has a null check, so the script still runs if a joint has not been assigned.

When the timer reaches zero, ResetPose restores every joint to its cached rotation. OnDisable also calls ResetPose, so the figure is not left in a mid-swing pose if the object is switched off. Pressing the button again sets the timer back to 2.4 seconds, so the walk can be replayed or restarted at any time. The speed and size of the walk can be changed in the Inspector with duration, swingAngle and cyclesPerSecond

Student2MinifigureWalk.cs | CarrotPickup.cs | CarrotCounter.cs | MouseInput.cs

Assembly-CSharp | Student2MinifigureWalk | CacheRestPose()

```
using UnityEngine;

Unity Script (1 asset reference) 0 references
public class Student2MinifigureWalk : MonoBehaviour
{
    Unity Serialized Field = Left hip | 6 references
    public Transform leftHip;
    Unity Serialized Field = Right hip | 6 references
    public Transform rightHip;
    Unity Serialized Field = Left shoulder | 6 references
    public Transform leftShoulder;
    Unity Serialized Field = Right shoulder | 6 references
    public Transform rightShoulder;

    Unity Serialized Field = 2.4 | 2 references
    public float duration = 2.4f;
    Unity Serialized Field = 30 | 1 reference
    public float swingAngle = 30f;
    Unity Serialized Field = 2 | 1 reference
    public float cyclesPerSecond = 2f;

    3 references
    private Quaternion leftHipRest;
    3 references
    private Quaternion rightHipRest;
    3 references
    private Quaternion leftShoulderRest;
    3 references
    private Quaternion rightShoulderRest;
    6 references
    private float timeLeft;
    3 references
    private bool poseCached;

    Unity Message | 0 references
    void Awake()
    {
        CacheRestPose();
    }

    public void PlayWalk()
    {
        if (!poseCached)
            CacheRestPose();

        timeLeft = duration;
    }

    Unity Message | 0 references
    void Update()
    {
        if (timeLeft <= 0f)
            return;

        timeLeft = Mathf.Max(0f, timeLeft - Time.deltaTime);

        float elapsed = duration - timeLeft;
        float swing = Mathf.Sin(elapsed * cyclesPerSecond * Mathf.PI * 2f) * swingAngle;

        if (leftHip) leftHip.localRotation = leftHipRest * Quaternion.Euler(swing, 0f, 0f);
        if (rightHip) rightHip.localRotation = rightHipRest * Quaternion.Euler(-swing, 0f, 0f);
        if (leftShoulder) leftShoulder.localRotation = leftShoulderRest * Quaternion.Euler(-swing, 0f, 0f);
        if (rightShoulder) rightShoulder.localRotation = rightShoulderRest * Quaternion.Euler(swing, 0f, 0f);

        if (timeLeft == 0f)
            ResetPose();
    }

    2 references
    void CacheRestPose()
    {
        if (leftHip) leftHipRest = leftHip.localRotation;
        if (rightHip) rightHipRest = rightHip.localRotation;
        if (leftShoulder) leftShoulderRest = leftShoulder.localRotation;
        if (rightShoulder) rightShoulderRest = rightShoulder.localRotation;

        poseCached = true;
    }
```


Unity 6
DH
Asset Store
AI
Unity VCS
Layout
Hierarchy
Scene
Asset Store
Game
Animator
Inspector
Name
RabbitSurvivalLEGO
Student2Minifigure
Left hip
Leg
Foot
Right hip
Left shoulder
Right shoulder
Torso
Head
Stud
Left eye
Right eye
Smile
Student2BrickMaterial
Student2BrickMaterial (2)
Student2BrickMaterial (1)
Rabbit Survival | LEGO Woodland
CarrotGameManager
Student2WalkingMinifigure
Persp
Student2Minifigure
Static
Tag
Untagged
Layer
Default
Transform
Position
Rotation
Scale
Student 2 Minifigure Walk (Script)
Script
Student2MinifigureWalk
Left Hip
Left hip (Transform)
Right Hip
Right hip (Transform)
Left Shoulder
Left shoulder (Transform)
Right Shoulder
Right shoulder (Transform)
Duration
2.4
Swing Angle
30
Cycles Per Second
2
Add Component
Project
Console
Favorites
Assets
RabbitSurvivalLEGO
EastWoodland
Materials
Meshes
Student2
Assets > RabbitSurvivalLEGO > Student2
Student2Minifigure
Student2MinifigureWalk
Student2Walk
Student2WalkController
Walk

Carrots, counter and HUD

The rabbit collects five carrots along the trail. Each carrot disappears when the rabbit touches it, and a counter on the screen shows how many have been collected. The feature uses two scripts: CarrotPickup.cs on each carrot and CarrotCounter.cs on a manager object.

Each carrot has a trigger collider and a CarrotPickup component. OnTriggerEnter runs when another collider enters the trigger. The script recognises the rabbit by checking whether the other object has a CharacterController, which is the component Student 3's rabbit controller uses for movement. This means no tag is needed, and other objects such as hunters or bricks cannot collect a carrot. A private collected flag stops the carrot from being counted twice. On a valid touch, the script sets the flag, calls CollectCarrot on the counter and hides the carrot with SetActive(false).

CarrotCounter has a public TMP_Text reference, counterText, which points to a TextMeshPro element on the Canvas, and a public totalCarrots value set to 5. Each carrot's counter field is assigned to this object in the Inspector. CollectCarrot increases the private collected value with Mathf.Min(collected + 1, totalCarrots), so the count can never go above the total. UpdateCounter writes the text in the format "Carrots: 0 / 5". It runs once in Start, so the HUD shows the starting count as soon as the scene loads, and again after every pickup. The text stays on the screen throughout, so the player can see progress without leaving the game view. When the fifth carrot is collected, the counter stops at "Carrots: 5 / 5".

The count is stored only in the running scene. When Play Mode stops, collected returns to zero and the disabled carrots become active again, so each play session starts with five carrots and a counter of 0 / 5. The pickup script has no victory or reward logic. The counter is for feedback and encourages the player to explore the trail.

[[CarrotCounter.cs]] • Carrot trigger and counter

Student2MinifigureWalk.cs | CarrotPickup.cs | **CarrotCounter.cs** | MouseInput.cs

Assembly-CSharp | CarrotCounter | totalCarrots

```
using TMPro;
using UnityEngine;

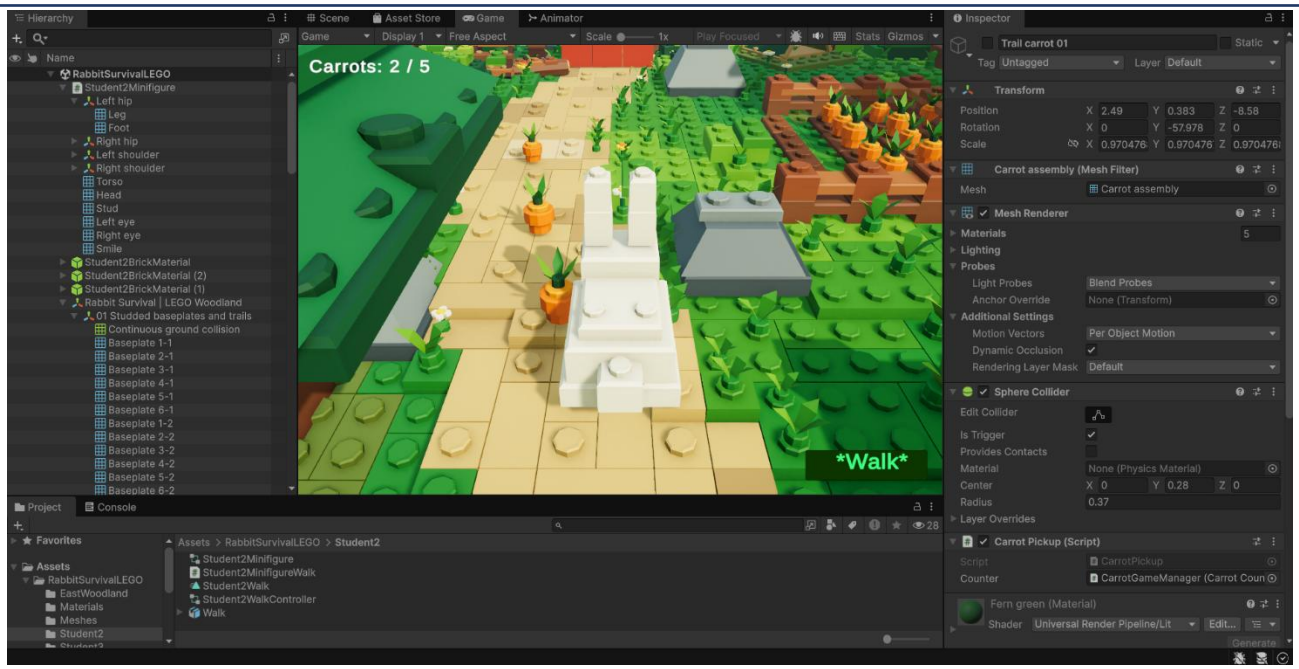
Unity Script (1 asset reference) | 1 reference
public class CarrotCounter : MonoBehaviour
{
    Unity Serialized Field = CarrotCounterText | 1 reference
    public TMP_Text counterText;
    Unity Serialized Field = 5 | 2 references
    public int totalCarrots = 5;

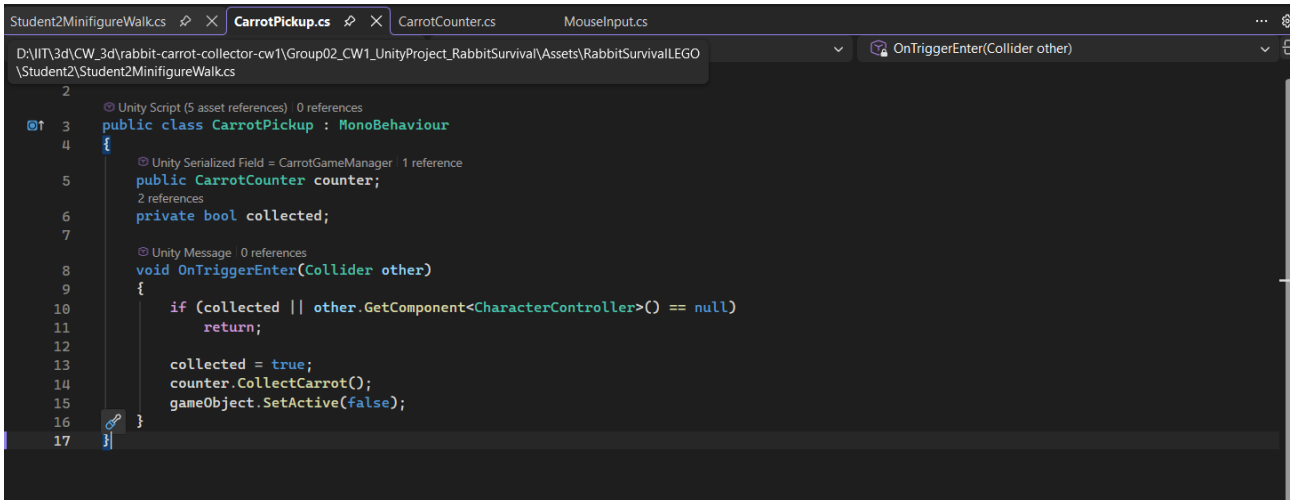
    3 references
    private int collected;

    Unity Message | 0 references
    void Start()
    {
        UpdateCounter();
    }

    1 reference
    public void CollectCarrot()
    {
        collected = Mathf.Min(collected + 1, totalCarrots);
        UpdateCounter();
    }

    2 references
    void UpdateCounter()
    {
        counterText.text = $"Carrots: {collected} / {totalCarrots}";
    }
}
```





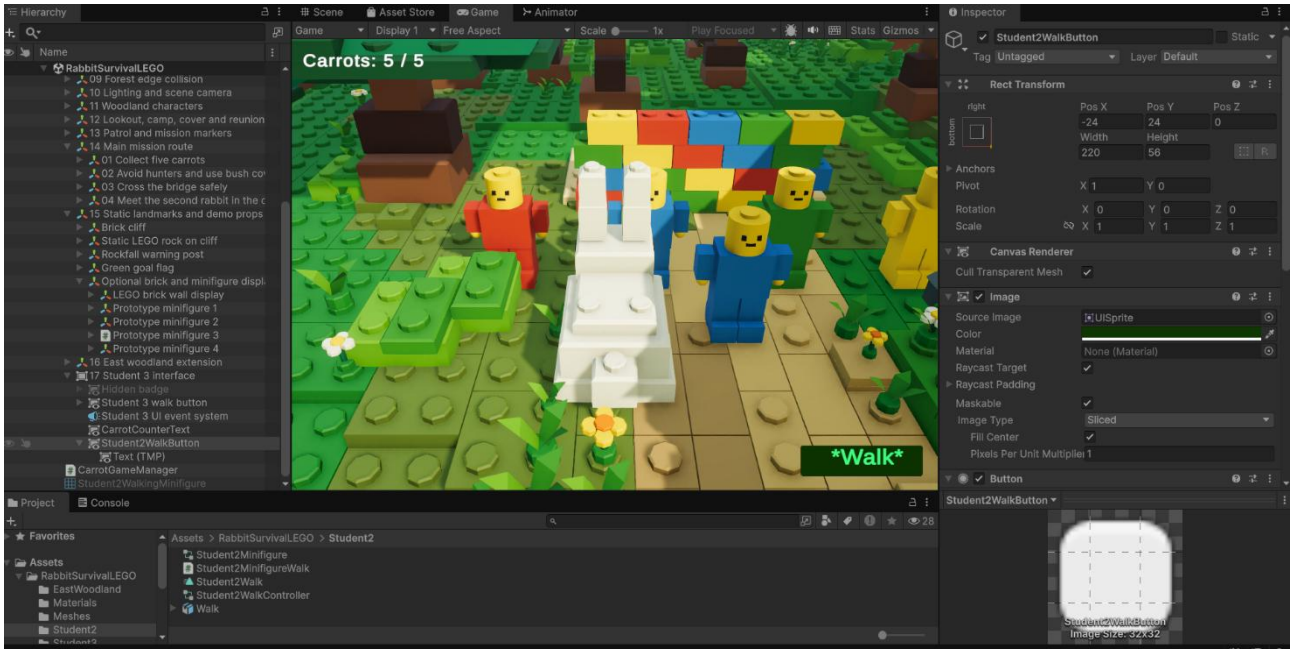
```
Student2MinifigureWalk.cs | CarrotPickup.cs | CarrotCounter.cs | MouseInput.cs
D:\IIT\3d\CW_3d\rabbit-carrot-collector-cw1\Group02_CW1_UnityProject_RabbitSurvival\Assets\RabbitSurvivalLEGO\Student2\Student2MinifigureWalk.cs
OnTriggerEnter(Collider other)

2
    Unity Script (5 asset references) | 0 references
3   public class CarrotPickup : MonoBehaviour
4   {
        Unity Serialized Field = CarrotGameManager | 1 reference
5       public CarrotCounter counter;
        2 references
6       private bool collected;
7
        Unity Message | 0 references
8       void OnTriggerEnter(Collider other)
9       {
10          if (collected || other.GetComponent<CharacterController>() == null)
11              return;
12
13          collected = true;
14          counter.CollectCarrot();
15          gameObject.SetActive(false);
16      }
17  }
```

02. Group integration and playtesting

Building the group scene

The imports went smoothly, and the only problem I had to fix was in the animation. The prototype minifigures from the other packages had to be animated by hand, because their joints were not set up for the walk. I added the Student2MinifigureWalk script to the figure and assigned its hip and shoulder joints in the Inspector. After that, the walk button worked in the group scene.



How my features connect to the group's work

The mouse bricks form the wall that blocks the trail. The rabbit cannot walk through it or around it, so the player must drag the bricks aside before the rabbit can continue. This connects my feature to Student 3's rabbit controller, since the rabbit and the bricks both use colliders. The carrot pickups also work with Student 3's controller. They recognize the rabbit by its CharacterController, so no extra tags or changes to the rabbit were needed. The carrot counter is a separate text element on the screen. The minifigure walk button works on its own and does not affect the other systems.



Recorded Play Mode Checks

I tested each feature in Play Mode after importing all the packages. I dragged a brick from the wall with the mouse, and it moved across the ground and stayed where I left it. The rabbit could then pass through the gap. When I pressed the walk button, the minifigure's arms and legs swung and then returned to the rest pose. Pressing the button again replayed the walk. I walked the rabbit through all five carrots along the trail. Each carrot disappeared on contact, and the counter increased from 0 / 5 to 5 / 5 and stopped there. When I stopped Play Mode and started it again, the carrots returned and the counter was back at 0 / 5.




Carrots: 2 / 5
Walk

03. Evaluation and AI use

Evaluation and further testing

The strongest part of my design is that the mouse brick is simple and direct. The player drags the brick they can see, and it moves on the ground in the same direction as the mouse. Because OnMouseDrag uses the brick's own collider, selecting and moving happen in one action, with no menu or extra key press. The wall also gives the rabbit a clear obstacle with an obvious solution, since the rabbit cannot walk through or around the bricks. The carrot counter works well for the same reason. The text stays on the screen, and "Carrots: 3 / 5" tells the player how far along they are without extra instructions.

There are three things I would like to improve. First, the bricks give no visual feedback. A highlight when the mouse is over a brick would show the player which objects can be moved. Second, transform.Translate moves the bricks directly instead of through the Rigidbody, so a dragged brick can push through other bricks or walls. Using Rigidbody.MovePosition would respect collisions. Third, the drag speed depends on the mouse movement multiplied by Time.deltaTime, so it can change with the frame rate. Removing Time.deltaTime from the mouse input would make it more consistent. The carrots would also benefit from a sound or effect on pickup, and a message when the count reaches 5 / 5.

The main problem during integration was the minifigures. Their joints were not set up for the walk, so I wrote a script that rotates the hips and shoulders and assigned the joints in the Inspector. Importing the packages one at a time, as described in section 02, meant I could check the scene after each import and find any problem quickly.

These choices connect to the 3DUI theory from the lectures. Dragging a brick is direct selection and manipulation. The player picks an object with the mouse and moves it, with a close match between the control and the result. The carrot pickups are triggers, because the rabbit's collider starts an event without any button press. The counter and the walk animation provide feedback, because they show the player that their action had an effect. For CW2, I would add highlight feedback to the bricks, move them through the Rigidbody, and connect the carrot counter to a goal or reward in the final scene.

AI assistance and evidence

I used ChatGpt and Claude to help me learn how to do several parts of my work: how to animate the minifigure, how to add UI buttons in Unity, how to set up an animation clip, how the carrot counter works, and general scripting help. I also used AI to validate and confirm the scripts, rechecking the settings and adjustments and correcting them. I tested all the results in Unity myself.